

SmartEdu: Comprehensive System Architecture & Engineering Report

1. Executive Summary

SmartEdu is a modern, AI-driven Learning Management System (LMS) designed to automate and enhance the educational experience for both educators and learners. By bridging traditional course management with advanced Large Language Models (LLMs), the platform automates administrative and pedagogical overhead—such as transcribing lectures, generating study guides, and creating dynamic assessments. The architecture follows a strict decoupled approach, utilizing a RESTful Python/Flask backend, a heavily normalized relational MySQL database, and asynchronous workflow automation to deliver real-time AI features without bottlenecking the core server.

2. Technology Stack & Architecture

The system is built on a scalable, modular technology stack designed for rapid local development and seamless future cloud deployment.

Backend Application Server

- **Framework:** Python 3 with **Flask** (v3.0.3).
- **Architecture:** Modular "Blueprint" design pattern. Routes are logically separated into domains (auth.py, courses.py, lessons.py, quizzes.py, etc.) ensuring single-responsibility principles and maintainability.
- **Authentication:** Stateless authentication utilizing **JSON Web Tokens (JWT)** via flask-jwt-extended. Route protection is strictly enforced using custom middleware decorators (@teacher_required, @student_required).
- **Security:** Password hashing is managed securely using bcrypt (cost factor 12) to prevent plaintext credential leaks.

Database Layer

- **Engine:** **MySQL 8.0** (Relational Database Management System).
- **ORM:** **SQLAlchemy** (via flask-sqlalchemy), providing a secure abstraction layer against SQL injection while managing complex database migrations and relationships.

- **Schema Design:** Heavily normalized (3NF) relational structure comprising 9 interconnected tables with strict cascading foreign key constraints to maintain data integrity.

Artificial Intelligence & Automation

- **Primary LLM: Google Gemini 1.5 Pro** via the Google AI Studio REST API.
 - **Automation Engine: n8n** (Node-based workflow automation). Used to orchestrate multi-step, asynchronous background tasks (e.g., API chaining for transcript fetching and AI processing) independent of the Flask server's request-response cycle.
-

3. Core Functional Modules

3.1. Role-Based Access Control (RBAC) & Authentication

The platform enforces strict role isolation between two primary user types:

- **Teachers:** Granted elevated privileges to execute CRUD (Create, Read, Update, Delete) operations on courses, lessons, and supplementary materials.
- **Students:** Granted read-only access to course materials, write access to personal progress trackers, and execution rights for quiz generation and submission.
- **Mechanism:** Upon successful login, the server issues a JWT containing the user's ID and role payload. This token is passed in the Authorization header for all subsequent API requests.

3.2. Asynchronous AI Summarization Pipeline (n8n)

To prevent the Flask server from timing out during heavy AI workloads, video summarization is offloaded to an asynchronous event-driven pipeline.

1. **Trigger:** A teacher inserts a new YouTube URL via the Flask `/api/lessons` endpoint.
2. **Webhook:** Flask instantly fires a non-blocking POST request to the local n8n webhook listener (`http://localhost:5678/webhook/smartedu-summary`).
3. **Extraction:** The n8n code node parses the YouTube URL via regex `((?:v=|youtu\.beV)([w-]{11})/)` to isolate the unique video ID.
4. **Transcription:** n8n calls a dedicated API (`yt-transcript-api`) to extract the video's closed captions, concatenating the JSON response into a flat, 3000-word text string.

5. **AI Processing:** The transcript is formatted into a zero-shot prompt and sent to the **Gemini 1.5 Pro** endpoint, instructed to return a concise, 5-8 sentence flowing prose summary.
6. **Callback:** n8n pushes the generated text back to Flask via the `/api/summaries/webhook` endpoint, which upserts the data into the `AI_Summaries` table.

3.3. Synchronous AI Quiz Generation

Students can test their knowledge on-demand via dynamic assessments.

1. **Context Retrieval:** When a student hits `/api/quizzes/generate/<lesson_id>`, Flask queries the database for the lesson's title and its pre-generated AI summary.
2. **Prompt Engineering:** The server constructs a strict system prompt demanding exactly 5 multiple-choice questions (A/B/C/D) of varying difficulty, formatted strictly as a JSON array.
3. **LLM Execution:** A synchronous call is made to the Gemini API (`requests.post`).
4. **Data Parsing & Storage:** The server sanitizes the response (stripping markdown fences), parses the JSON array, and sequentially inserts the parent Quiz record and the 5 child QuizQuestion records into the database.
5. **Evaluation:** Answers are tracked via the `/api/quizzes/<quiz_id>/submit` endpoint, which calculates the raw score, updates the Boolean `is_correct` flags, and calculates the final percentage.

3.4. Progress Tracking & Analytics

- **Status States:** Tracks user progression through an ENUM state machine (not-started, in-progress, completed).
- **Statistical Aggregation:** The `/api/progress/stats` endpoint utilizes SQLAlchemy aggregation functions (`func.count`, `func.avg`) to dynamically calculate real-time completion rates and average quiz scores across the student's entire enrollment history.

4. Database Entity-Relationship Modeling

The database satisfies advanced relational design rubrics, utilizing complex joins, unique constraints, and aggregate functions.

Core Entities & Relationships

1. **Users:** The central entity. One-to-Many relationships with Courses (Teacher), Lesson_Materials (Teacher), Progress_Tracking (Student), and Quizzes (Student).
2. **Courses:** Parent entity for educational content. Owned by a Teacher.
3. **Enrollments:** A junction/mapping table resolving the Many-to-Many relationship between Users (Students) and Courses. Enforces a composite unique key to prevent duplicate enrollments.
4. **Lessons:** Child of Courses. Stores the critical youtube_url.
5. **AI_Summaries:** One-to-One relationship with Lessons. Contains the large text payload generated by the n8n pipeline.
6. **Lesson_Materials:** One-to-Many relationship with Lessons. Stores supplementary data (slides, notes) utilizing a material type ENUM.
7. **Progress_Tracking:** A junction table tracking the intersection of a specific User and a specific Lesson.
8. **Quizzes:** Child of Lessons and Users. Tracks an individual test attempt and final score.
9. **Quiz_Questions:** Child of Quizzes. Stores the AI-generated questions, options, the correct answer, and the student's submitted answer.

Advanced Constraints Applied

- **ON DELETE CASCADE:** Applied universally to foreign keys. Deleting a Course recursively wipes its Lessons, Quizzes, Summaries, and Materials to prevent orphaned data.
- **UNIQUE KEY:** Applied to junction tables (e.g., student_id + lesson_id in Progress Tracking) to prevent redundant state entries.

5. Future Scalability & Enhancements

While the current MVP successfully achieves its technical requirements, the architecture allows for seamless scaling:

1. **Cloud Migration:** Transitioning the local MySQL instance to a managed database (e.g., AWS RDS or Google Cloud SQL) and Dockerizing the Flask application for deployment on an elastic container service.
2. **Vector Embeddings (RAG):** Upgrading the quiz generator by storing lesson transcripts as vector embeddings in a database like Pinecone. This would allow the AI to generate highly specific questions by performing semantic search over hours of video content, rather than relying solely on the short text summary.
3. **WebSockets:** Implementing WebSockets (via Flask-SocketIO) to push a real-time notification to the frontend the exact moment the n8n background summary pipeline completes its task.